BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**Technical Documentation**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2026-03-01	S. Köhne	Stable	Initial Release: ProTU Enterprise Compliance Guide

1. Product Overview: Process Trust Unit (ProTU)

The **ProTU (Process Trust Unit)** is the official certification and audit tool provided by BrainAI UG for the ProSuite Industrial ecosystem.

In high-security environments, it is simply not enough to trust that cryptographic software "works." Hardware deviations, faulty operating system updates, or silent RAM corruption can severely compromise key integrity. ProTU solves this critical problem: It executes deep cryptographic penetration, entropy, and memory allocation tests directly on the customer's target system, proving the correct operation of the entire suite mathematically and physically.

Deployment Scenarios for Enterprise Customers:

- **Commissioning:** One-time execution after installing the ProSuite on new server infrastructure.
- **Continuous Compliance:** Automated execution (e.g., via cronjob or task scheduler) for continuous monitoring required by ISO 27001 / TISAX audits.
- **Forensics:** Deep-level checking for "stuck-at" faults or compromised entropy sources within the OS.

2. Operation and Execution (CLI)

ProTU is delivered as a standalone, zero-dependency portable executable (`protu_industrial.exe` for Windows, `protu_industrial` for Linux). It requires no installation and runs autonomously in the user-space.

2.1 Standard Audit (Full System)

To audit the entire system, call the application without parameters or explicitly pass the `ALL` parameter.

Windows:

```
> protu_industrial.exe ALL
```

Linux:

```
$ ./protu_industrial ALL
```

Note: This command systematically tests all licensed user-space modules (ProKey, ProHash, ProED, ProEDC, ProChat) as well as the framework's own integrity.

2.2 Targeted Module Audits

For isolated security checks or targeted CI/CD testing, a specific module name can be passed as an argument:

Command	Audit Target & Scope
`protu_industrial ProKey`	**Hardware Entropy:** Checks the random number generator for monobit bias, repetitions, and physical starvation.
`protu_industrial ProHash`	**Integrity:** Validates the sponge engine, avalanche effect, and collision resistance.
`protu_industrial ProEDC`	**Encryption:** Verifies AEAD encapsulation, zero-trust memory wiping, and boundary checks.
`protu_industrial WASM`	**Web Vault:** Verifies the memory safety and overflow protection of the WebAssembly heap for Node.js backends.
`protu_industrial ProKM`	**Kernel (Ring-0):** *Optional.* Checks the Windows/Linux driver for memory leaks and direct I/O security.

3. The Audit Report (.brep) & Dual-Seal Technology

After each run, ProTU generates a highly detailed log file in the execution directory. This file strictly follows the naming convention `ProTU_YYYYMMDD_MODULE.brep` (e.g., `ProTU_20260301_ALL.brep`).

3.1 Report Structure

The report chronologically lists every executed test, the injected payload, and the exact cryptographic result. The results are classified into three severity levels:

- **[INFO]**: Standard status message or the initialization of a test routine.
- **[PASS]**: The test was passed successfully. The module operates mathematically and physically as specified.
- **[FAIL] / [CRITICAL]**: A security test failed. *The system must be isolated immediately to prevent data leaks.*

3.2 Dual-Seal (Report Integrity)

An audit report is only reliable if it is mathematically impossible to manipulate it after the fact. Therefore, ProTU utilizes our proprietary **Dual-Seal Technology**.

Before the `.brep` file is written to the disk and closed, ProTU calls the internally statically linked `ProHash` engine. A deterministic SHA-256 (ProHash) fingerprint is calculated over the entire report text and appended as a cryptographic seal at the bottom of the file.

- **Benefit for the Auditor:** If a malicious actor or malware alters even a single character in the report (e.g., changing a `FAIL` to a `PASS`), the cryptographic seal breaks instantly, rendering the report demonstrably invalid for any auditor.

4. Interpretation of Exit Codes (For Automation)

For seamless integration into enterprise management and monitoring systems (e.g., Nagios, PRTG, Jenkins), ProTU returns standardized system exit codes:

- **Exit Code 0 (Success):** All executed tests were passed (`g_pt_failed == 0`). The system is ready for production and "Gold Standard" certified.
- **Exit Code 1 (Failure):** At least one security-critical test failed. The operation was aborted immediately, and the fatal error is detailed in the `.brep` report.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**